ROS 2 REPORT

Siddharth Prasad
Department of Computer Science Engineering
MIT Manipal,India
siddharthprasad61@gmail.com

***Abstract*—The Robot Operating System 2 (ROS 2) is an open-source middleware framework that provides the tools, libraries, and communication infrastructure needed to build modern robotic applications. This paper presents an overview of ROS 2, covering its DDS-based architecture, its core communication mechanisms (nodes, topics, services, and actions), and the improvements it offers over the original ROS, such as real-time support, enhanced security, and multi-robot scalability. The paper also discusses how ROS 2 is applied in areas like autonomous vehicles, industrial automation, and mobile robotics, and highlights the role of version control tools such as Git in managing collaborative ROS 2 development.**

I. Introduction

The Robot Operating System 2 (ROS 2) is an open-source middleware framework designed for developing robotic applications. It is the successor to the original ROS and was redesigned to meet the demands of modern robotics, including real-time performance, multi-robot coordination, and deployment in production environments. ROS 2 is built on the Data Distribution Service (DDS) standard, which provides reliable, decentralized communication between software components called nodes through mechanisms such as topics, services, and actions. Unlike its predecessor, it removes the dependence on a central master node, improves security and scalability, and supports multiple platforms, including Linux, Windows, and macOS, as well as embedded systems. Its modular architecture, extensive library of tools, and active community make it widely used in areas such as autonomous vehicles, industrial automation, drones, and mobile robotics research.

A. What is ROS 2?

ROS 2 (Robot Operating System 2) is not an operating system in the traditional sense. It is a set of open-source software libraries, tools, and communication standards that help developers build robot applications. It lets a robot's software be split into small, independent programs called nodes, each handling a specific task such as reading a sensor, planning a path, or controlling a motor. These nodes exchange data through topics (continuous data streams), services (request-response calls), and actions (long-running tasks with feedback), all built on the DDS communication standard. ROS 2 also provides ready-made tools for visualization, simulation, and debugging, such as RViz and Gazebo, which makes it easier to develop, test, and reuse robotic software across different hardware platforms.

II. ROS 2 Computation Graph

- Definition: The ROS 2 computation graph (also called the ROS graph) is the network of all ROS 2 elements that are processing data together at a given moment. It shows which nodes are running, how they are connected, and what data flows between them. Nodes are the vertices of the graph, and the communication links (topics, services, actions) are the edges. The graph is dynamic: it changes whenever a node starts, stops, publishes, or subscribes.
- Elements of the Graph:
  1) Nodes:Independent processes that perform computation
  2) Topics: Named channels for publish/subscribe data streams
  3) Messages:Typed data structures sent over topics
  4) Services: Request/response links between client and server nodes
  5) Actions:Goal/feedback/result links for long-running tasks
  6) Parameters:Configuration values stored inside nodes
  7) Namespaces and names:Organize and uniquely identify graph element
  8) Discovery layer (DDS):Lets nodes find each other automatically
  9) Domain ID:Separates independent graphs on the same network
- How the Graph Is Formed? Unlike ROS 1, ROS 2 has no central master. The graph is formed in a decentralized way by DDS discovery: When a node starts, it announces itself (its name, publishers, subscribers, services, and clients) to the network.Other nodes in the same domain hear this announcement and announce themselves in return. Publishers and subscribers with the same topic name, message type, and compatible QoS are matched. Data then flows peer-to-peer between the matched nodes. When a node shuts down or loses liveliness, it is removed from the graph and the connections are dropped.
- Domain ID: The ROS DOMAIN ID environment variable (0 to 101 are recommended, with 0 as the default) splits the network into separate graphs. Nodes with the same domain ID discover each other, while nodes on different IDs are isolated. This is useful when several robots or students share one network.
- Inspecting the Graph:

1) ros2 node list
2) ros2 node info ¡node¿
3) ros2 topic list -t
4) ros2 topic info ¡topic¿ –verbose
5) ros2 service list -t
6) ros2 action list -t
7) ros2 param list
8) rqt graph

III. NODES

- Definition: A node is the fundamental building block of a ROS 2 application. It is a single, independent process that performs one specific task, such as reading a camera, processing LiDAR data, planning a path, or driving a motor. A complete robot system is made up of many nodes working together and communicating with one another. This modular approach makes software easier to develop, debug, reuse, and maintain, because a fault in one node does not necessarily bring down the whole system.
- Key Characteristics:
  1) Single purpose: each node should do one job well.
  2) Independent: nodes can be started, stopped, or replaced without restarting the whole system.
  3) Independent: nodes can be started, stopped, or replaced without restarting the whole system.
  4) Distributed: nodes can run on the same computer or on different machines across a network.
  5) Language-independent: nodes can be written in C++ (rclcpp), Python (rclpy), and other languages, and still communicate with each other.
  6) Discoverable: nodes find each other automatically through DDS, with no central master (unlike ROS 1).
  7) Uniquely named: each node has a name and can be placed in a namespace to avoid conflicts, for example /robot1/camera node.
- Communication Mechanisms: Nodes exchange information in four main ways:
  1) Topics (publish/subscribe): one-way, continuous data streams. A publisher node sends messages on a named topic, and any number of subscriber nodes receive them. Example: a camera node publishes images to /camera/image raw.
  2) Services (request/response): synchronous, two-way communication. A client sends a request and waits for the server's response. Example: a node asks another to reset the odometry.
  3) Actions (goal/feedback/result): for long-running tasks. A client sends a goal, receives periodic feedback, and gets a final result. The goal can also be canceled. Example: "navigate to point X."
  4) Parameters: configuration values that belong to a node, such as speed limits or sensor rates, and can be changed at runtime without recompiling.
- Node Lifecycle (Managed Nodes) ROS 2 offers an optional lifecycle node with a defined state machine, giving more control over startup and shutdown. The primary states are:
  1) Unconfigured: the node exists but is not set up.
  2) Inactive: configured but not processing data.
  3) Active: fully running and processing data.
  4) Finalized: shut down and ready to be destroyed.
- Advantages of the Node-Based Design:
  1) Modularity and code reuse across projects.
  2) Fault isolation, since one failing node doesn't crash the rest
  3) Easy testing of individual components
  4) Scalability from a single robot to multi-robot systems
  5) Flexibility to mix programming languages and hardware
- Some Command-Line Tools:
  1) ros2 run <package><executable >: For Starting a node
  2) ros2 node list: Lists all the running nodes
  3) ros2 node info <node name>: Show a node's publishers, subscribers, services, and actions
  4) ros2 topic list / ros2 topic echo ¡topic¿:Inspect topics and their messages.
  5) ros2 param list / ros2 param get ¡node¿ ¡param¿: View node parameters
  6) ros2 run rqt graph rqt graph :Visualize nodes and their connections

IV. TOPICS

- Definition:A topic is a named communication channel that nodes use to exchange data continuously. It follows the publish/subscribe model: a node that produces data (the publisher) sends messages to a topic, and any node interested in that data (a subscriber) receives them. Publishers and subscribers do not know about each other directly; they only need to agree on the topic name and the message type. This decoupling makes topics the most commonly used communication method in ROS 2.
- Key Characteristics:
  1) Anonymous and decoupled: publishers don't know who is listening, and subscribers don't know who is sending.
  2) Many-to-many: a topic can have multiple publishers and multiple subscribers at the same time.
  3) Unidirectional: data flows only from publisher to subscriber.
  4) Asynchronous: the publisher does not wait for any reply.
  5) Strongly typed: every topic carries exactly one message type, for example sensor msgs/msg/Image.
  6) Named with a hierarchy: topic names look like paths, such as /robot1/camera/image raw, and can be placed in namespaces.

7) Best suited for streaming data: sensor readings, odometry, velocity commands, and robot state.

- How Topics Work: A publisher node creates a publisher for a topic with a given message type. A subscriber node creates a subscription to the same topic name and type. DDS discovery automatically connects them, with no central master. Each time the publisher sends a message, DDS delivers it to every matching subscriber, which processes it in a callback function.
- Messages: Data sent over topics is defined by message types, described in .msg files. A message is a structure of typed fields. Example of a custom message Position.msg: float64 x, float64 y, float64 theta.
- Common built-in message packages:
  1) std msgs: String, Int32, Float64, Bool
  2) geometry msgs:Twist, Pose, Point, Transform
  3) sensor msgs: Image, LaserScan, Imu, PointCloud2
  4) nav msgs: Odometry, Path, OccupancyGrid
- Some Command-Line Tools:
  1) ros2 topic list: List all active topics
  2) ros2 topic list -t: List topics with their message types
  3) ros2 topic echo <topic >: Print messages as they arrive
  4) ros2 topic info <topic >: Show type, publisher count, and subscriber count
- Advantages: Loose coupling between nodes. Easy to add new subscribers without changing the publisher. Efficient for high-frequency sensor data. Data can be recorded and replayed with rosbag. Works across machines and programming languages.

V. SERVICES

- Definition: A service is a communication mechanism that follows the request/response (client/server) model. A client node sends a request to a service server, the server processes it, and it sends back exactly one response. Unlike topics, which stream data continuously, services are used for short, on-demand interactions where the caller needs a result, such as asking a node to reset odometry, toggle a sensor, or compute something.
- Key Characteristics:
  1) Request/response: every request produces one response.
  2) Two-way: data flows from client to server and back.
  3) One server, many clients: a service name can have only one server, but any number of clients can call it.
  4) Strongly typed: each service has a type made of a request part and a response part.
  5) Named with a hierarchy: for example /add two ints or /robot1/reset odometry
  6) Best for quick tasks: services should finish fast, because they are not meant for long operations.
  7) Not for streaming: use topics for continuous data.
- How Services Work
  1) A server node creates a service with a name, a type, and a callback function.
  2) A client node creates a client for the same name and type.
  3) DDS discovery connects them automatically.
  4) The client sends a request.
  5) The server's callback runs, builds a response, and returns it.
  6) The client receives the response (in ROS 2 this is normally done asynchronously, so the node is not blocked).
- Service Type Definition: Service types are defined in .srv files. The request and response are separated by three dashes (—). Example AddTwoInts.srv:

```
int64 a
int64 b
---
int64 sum
```

- Common built-in service types:
  1) std srvs: Empty, Trigger, SetBool
  2) example interfaces: AddTwoInts
  3) rcl interfaces: GetParameters, SetParameters, ListParameters.
  4) nav msgs: GetMap, GetPlan
  5) lifecycle msgs : ChangeState, GetState
- Some Useful Command-Line Tools:
  1) ros2 service list: List all active services.
  2) ros2 service list -t: List services with their types.
  3) ros2 service type <service >: Show the type of a service
  4) ros2 service find <type >: Find services of a given type
  5) ros2 interface show <type >: Display request and response structure
  6) ros2 run ¡package¿ <server executable >:Start a service server

VI. CUSTOM INTERFACES

- Defintion In ROS 2, an interface is the definition of the data structure that nodes use to communicate. There are three kinds: messages (.msg) for topics, services (.srv) for request/response calls, and actions (.action) for long-running tasks. Standard interfaces such as std msgs and geometry msgs cover common needs, but when a robot needs a data structure that does not exist, you can define a custom interface of your own.
- Why Use Custom Interfaces:
  1) Standard types may not fit your data (for example, a battery status with voltage, temperature, and charge level).
  2) They make the code self-documenting, with meaningful field names.
  3) They avoid packing unrelated data into generic types like String.

4) They can be reused across packages and shared between teams.

- Supported Field Types: Primitive types: bool, byte, char, float32, float64, int8, uint8, int16, uint16, int32, uint32, int64, uint64, string, wstring
  1) Fixed-size: int32[3] values
  2) Unbounded dynamic: int32[] values
  3) Bounded dynamic: int32[¡=5] values
- Naming Rules:
  1) File names must be in CamelCase (for example, BatteryStatus.msg).
  2) Field names must be lowercase with underscores (for example, charge level).
  3) Constants must be in UPPERCASE.
- Useful Command-Line Tools:
  1) ros2 interface list : List all available interfaces
  2) ros2 interface package <pkg >: List interfaces in a package
  3) ros2 interface show <type >: Show the structure of an interface
  4) ros2 interface proto <type >:Show an example instance of the interface
  5) ros2 topic pub ¡topic¿ ¡type¿ '¡data¿': Publish a custom message
- Common Problems and Fixes:
  1) Import error (ModuleNotFoundError): you forgot to build or run source install/setup.bash.
  2) Build error about invalid names: file names must be CamelCase and field names lowercase.
  3) Missing dependency error: add DEPENDENCIES in CMakeLists.txt and ¡depend¿ in package.xml for any nested standard types
  4) Changes not appearing: after editing a .msg, .srv, or .action, rebuild the interface package and every package that depends on it.
  5) Python-only package: the interfaces still must live in an ament cmake package.

VII. COLCON

- Definition: colcon (short for collective construction) is the command-line build tool used in ROS 2 to build, test, and install multiple packages in a workspace with a single command. A typical ROS 2 project is made of many packages that depend on each other (nodes, custom interfaces, launch files, and so on). colcon works out the dependency order, builds each package with the right build system, and puts the results into one shared location so they can be used together. It replaced the ROS 1 tools catkin make and catkin build.
- Why colcon Is Needed:
  1) ROS 2 packages can use different build systems (CMake, Python setuptools), and colcon handles all of them.
  2) It builds packages in the correct order based on their dependencies.
  3) It builds independent packages in parallel, which saves time.
  4) It provides one consistent interface for building, testing, and installing.
  5) It is not limited to ROS 2 and can build any package that follows a supported build system.
- Key Characteristics:
  1) Meta build tool: it does not compile code itself; it calls the underlying build system (ament cmake, ament python, or plain cmake).
  2) Workspace-based: it works on a folder containing a src directory.
  3) Dependency-aware: it reads package.xml files to determine build order.
  4) Parallel builds: multiple packages are built at the same time.
  5) Out-of-source builds: build artifacts are kept separate from your source code.
  6) Extensible: supports plugins, mixins, and custom verbs.
- Common colcon Commands:
  1) colcon build: Build all packages in the workspace
  2) colcon build –packages-select <pkg >: Build only the named package
  3) colcon build –packages-up-to <pkg >: Build a package and everything it depends on
  4) colcon build –packages-above <pkg >:Build a package and everything that depends on it
  5) colcon build –symlink-install: Link files instead of copying them
  6) colcon build –parallel-workers <N>: Limit the number of parallel jobs
  7) colcon test: Run tests for all packages
  8) colcon test-result –verbose: Show test results
  9) colcon list : List packages found in the workspace
  10) colcon graph: Show the package dependency graph
- Common Problems and Fixes:
  1) Package not found after building: you forgot source install/setup.bash.
  2) Build run from the wrong folder: always run colcon build from the workspace root (ros2 ws), not from inside src.
  3) Strange errors after changing things: delete build/, install/, and log/ and rebuild.
  4) Changes not taking effect: rebuild the package, or use –symlink-install for Python and launch files.
  5) Dependency errors: run rosdep install and check package.xml for missing ¡depend¿ lines.
  6) Mixing sourced workspaces: build in a fresh terminal where only the intended workspaces are sourced.
  7) Very slow builds on small machines: use –parallel-workers 2 to reduce memory use.
  8) Warnings about setup.py install being deprecated: this comes from newer setup tools versions and is usually harmless for ROS 2 Python packages.

VIII. WORKSPACE STRUCTURE

- Definition: A workspace is a directory that contains ROS 2 packages and is used to build, install, and run them together. It is the standard way to organize your own robot code in ROS 2. A workspace is built with colcon, and the results are made available to your terminal by sourcing a setup script. You can have many workspaces on one computer, for example one per project.
- The Four Main Folders:
  1) src: Holds all source packages (your code, cloned repositories)
  2) build:Intermediate files, one subfolder per package, used during compilation
  3) install:Final executables, libraries, launch files, and setup scripts
  4) log:Logs of every build and test run, with timestamps
- Underlay and Overlay: Underlay: the base ROS 2 installation (for example /opt/ros/humble), sourced first. Overlay: your workspace, sourced on top of the underlay.You can stack several overlays. Packages in a later overlay take priority over packages with the same name in earlier ones. source /opt/ros/humble/setup.bash —>underlay source /ros2 ws/install/setup.bash —<overlay
- Useful Commands Line:
  1) colcon build
  2) colcon build –packages-select <pkg>
  3) colcon build –symlink-install
  4) colcon list
  5) ros2 pkg list
  6) ros2 pkg prefix <pkg>
  7) ros2 pkg executables <pkg>
- Rules and Tips:
  1) Do not nest one package inside another package.
  2) Each package needs a unique name within the workspace.
  3) Never edit files inside build/ or install/ (unless using –symlink-install).
  4) Do not build in a terminal where another overlay is sourced by mistake.
  5) Add a .gitignore with build/, install/, and log/, and commit only src/.
  6) Put an empty file named COLCON IGNORE in any folder you want colcon to skip.
  7) Keep one workspace per project to avoid dependency conflicts.
- Common Problems and Fixes:
  1) Package not found: run source install/setup.bash after building.
  2) Built from the wrong directory: run colcon only from the workspace root.
  3) Python changes not applied: rebuild or use –symlink-install.
  4) Launch or config file missing at runtime: you forgot to install the folder in setup.py or CMakeLists.txt.
  5) Strange build behavior: delete build/, install/, and log/, then rebuild.
- Advantages: Keeps projects organized and self-contained,Allows many packages to be built together with the correct dependency order,Overlays let you modify packages without touching the base installation,Easy to share through Git (only src is needed)

IX. QoS Settings.

- Definition: Quality of Service (QoS) is a set of policies that control how messages are delivered between publishers and subscribers. QoS comes from the DDS middleware that ROS 2 is built on. It lets you tune communication for different conditions, such as a fast but lossy Wi-Fi link, a sensor that streams at a high rate, or a map that must reach nodes that start late. ROS 1 had almost no such control, which is one of the main improvements in ROS 2.
- Why QoS Is Needed?
  1) Different data has different needs (a lidar scan can lose a message, a velocity command should not).
  2) Networks can be unreliable, and QoS defines how the system reacts.
  3) It controls memory use and latency through queue sizes and delivery guarantees.
  4) It makes late-joining nodes and real-time deadlines possible.
- Main Qos Policy Details:
  1) History and Depth
     - keep last stores the last N samples (N is the depth). Old samples are dropped when the queue is full.
     - keep all stores every sample, limited only by the middleware's resources.
     - A small depth saves memory, and a larger depth protects against bursts or slow subscribers.
  2) Reliability:
     - reliable: the middleware retransmits lost messages, so delivery is guaranteed (at the cost of extra latency and bandwidth).
     - best effort: messages are sent once, and lost ones are not resent. This is faster and suits high-rate sensor data.
  3) Durability:
     - volatile: no messages are kept for late joiners, so a new subscriber only gets messages published after it connects.
     - transient local: the publisher keeps the last messages (according to its history depth) and sends them to new subscribers. This is useful for maps, static transforms, and configuration data.
  4) Deadline: The publisher promises to publish at least once per deadline period, and the subscriber expects to receive data at that rate. If the deadline is missed,

a deadline event is triggered, which can be handled in code.

5) Lifespan: Each message gets an expiry time. Messages older than the lifespan are not delivered to subscribers, which prevents acting on stale data.
6) Liveliness and Lease Duration:
   - automatic: the system considers a node alive as long as any of its publishers is active.
   - manual by topic: the publisher must explicitly assert that it is alive
   - If a publisher fails to show it is alive within the lease duration, a liveliness-lost event is raised.

ROS QUESTION 1

A. Graph Components and Data Flow:

- Publisher Node: Represented by the left-most oval, this node serves as the data origin point. It handles user I/O operations, capturing the numerical inputs for angular velocity and wheel radius. It encapsulates these variables into the custom TaskOne message structure and continuously broadcasts the data to the network.
- Communication Topic: Represented by the central rectangle, this acts as the asynchronous communication bus. The directed arrow from the publisher indicates that data is being actively written to this specific channel. Utilizing a centralized topic ensures modularity, allowing the data stream to be accessed without direct point-to-point coupling.
- Subscriber Node: Represented by the right-most oval, this node functions as the data processor. The directed arrow originating from the topic indicates that the subscriber is actively listening to the channel. Upon receiving the custom message payload, the node triggers a callback function to extract the variables and compute the final longitudinal speed using the kinematic relationship $v = \omega \times r$

B. Data Synchronization via Custom Messages:

By bundling ang vel and radius into a single custom TaskOne.msg packet, you ensure both variables arrive at the subscriber at the exact same timestamp. If you published them on two separate standard topics, network delays could cause the speed calculation to use an old radius with a new angular velocity.

C. Node Decoupling for Modularity:

The publisher and subscriber are entirely blind to each other; they only interact with the /taskone communication bus. In a real autonomous vehicle, this architecture means you could physically swap out the wheel speed sensor (the publisher) or change its code entirely without ever needing to modify the physics algorithms (the subscriber).

D. The ament cmake Build Pipeline:

The system relies on the CMakeLists.txt and package.xml files to bridge your two independent packages. By defining wheel interfaces as a dependency, the ROS 2 compiler automatically translates your simple text message file into a complex C++ header file (task one.hpp) so your worker nodes can interact with the custom data structure natively.

X. ROS QUESTION 2:

A. Graph Components and Data Flow:

- String Publisher Node: Serving as the data origin point, this node publishes the user-inputted string directly to the /user strings topic.
- Processing and Routing Node: his node acts as the central hub of the network, functioning simultaneously as a subscriber and a publisher. It first subscribes to the /user strings topic to intercept the incoming string data. After processing the palindrome logic, it actively publishes an integer flag (1 or 0) forward to the /palindrome result topic. To complete the bidirectional loop, it also maintains an active subscription to the /feedback topic to listen for the final system response.
- Feedback Node: his node functions as the final evaluator. It subscribes to the /palindrome result topic to receive the integer flag from the central hub. It then translates this integer into a standard string and publishes that confirmation back to the /feedback topic, returning the data to

B. Hybrid Message Implementation:

The pipeline demonstrates the integration of both custom and standard ROS 2 message interfaces. While the initial input requires a custom compiled RT2String message, the subsequent communication utilizes standard library types (std msgs::msg::Int32 and std msgs::msg::String). Utilizing standard messages for basic binary flags (1/0) and

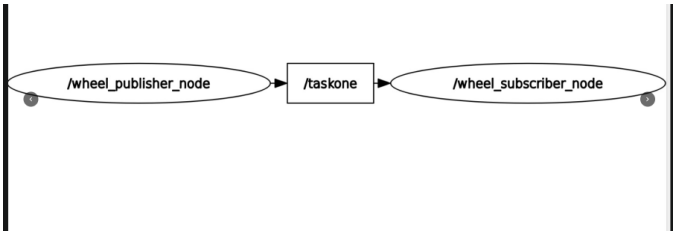


Fig. 1. ROS 2 computation graph 1

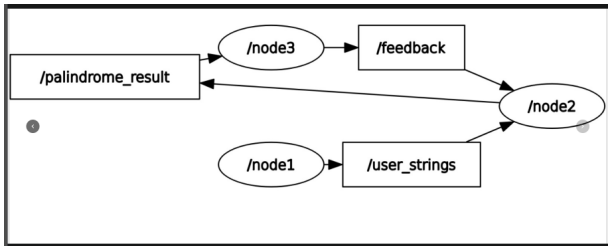


Fig. 2. ROS 2 computation graph 2

simple state confirmations (”Yes”/”No”) reduces unnecessary compilation overhead and maximizes compatibility with other built-in ROS 2 debugging tools.

C. Concurrent Asynchronous Callbacks:

Node 2 functions as a complex transceiver, acting simultaneously as a publisher and a subscriber. It relies on the rclcpp::spin() executor to manage an event-driven queue. This allows Node 2 to independently maintain two separate callback functions—one triggered by the /user strings topic and another triggered by the /feedback topic—without the node freezing or blocking while waiting for data on either channel.

D. Closed-Loop Feedback Architecture:

Unlike a unidirectional data stream, this three-node network establishes a closed-loop feedback system. Node 2 delegates a task (sending the integer flag to Node 3) and maintains an open listener to verify the result (receiving ”Yes” or ”No”). In autonomous vehicle software, such as the Formula Manipal Driverless stack, this architecture is critical for state verification (e.g., a central controller sending a steering angle command and waiting for a confirmation string from the physical actuator before proceeding).

E. Distributed Processing:

The palindrome verification logic is intentionally separated from the final string translation logic across Node 2 and Node 3. This modular, distributed approach prevents a single node from becoming computationally overloaded. It adheres to the ROS 2 best practice of assigning a single, specific responsibility to each node, ensuring the system remains highly scalable.

Conclusion

The successful implementation of these workflows highlights the robust, modular architecture of the Robot Operating System 2 (ROS 2) and its critical role in autonomous software development. By utilizing a publisher-subscriber middleware paradigm, the system achieves strict node decoupling, allowing processes to operate independently and communicate asynchronously. The integration of custom messages demonstrates the framework’s capability to synchronize complex data payloads, which is vital for real-time processing.

Furthermore, leveraging the `ament_cmake` build system ensures the seamless compilation of custom interfaces, while diagnostic tools like `rqt_graph` provide essential topological verification. Applying these core ROS 2 mechanics establishes the scalable, fault-tolerant software foundation necessary to manage the distributed subsystems onboard the Formula Manipal Driverless vehicle.

References

[1] Open Source Robotics Foundation, “ROS 2 Humble Hawksbill Documentation,” *ROS 2 Project*, 2022. [Online]. Available: https://docs.ros.org/en/humble/

[2] Open Source Robotics Foundation, “Writing a simple publisher and subscriber (C++),” *ROS 2 Humble Tutorials*. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Cpp-Publisher-And-Subscriber.html

[3] Open Source Robotics Foundation, “Creating custom msg and srv files,” *ROS 2 Humble Tutorials*. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Custom-ROS2-Interfaces.html

[4] Open Source Robotics Foundation, “Using rqt_console and rqt_graph,” *ROS 2 Humble Tutorials*. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Using-Rqt-Console/Using-Rqt-Console.html

[5] Canonical Ltd., “Ubuntu 22.04 LTS (Jammy Jellyfish) Release Notes,” *Ubuntu Documentation*, 2022. [Online]. Available: https://discourse.ubuntu.com/t/jammy-jellyfish-release-notes/24668